IE437 · DATA-DRIVEN DECISION MAKING AND CONTROL — LECTURE 1

Optimization Problem Modeling

Before you can learn a decision, you must be able to state one

Jinkyoo Park · KAIST

The atom every later method is built from

HANDOFF

The handoff — the purest decision

Lecture 0 drew the map. We start at the corner where nothing has been taken away yet.

Where we are — the simplest cell of the map

STAGES	MODEL	AGENTS
static	model-based	single agent

	MODEL-BASED	DATA-DRIVEN
Static, single	optimisation *(Lec 1)*	Bayesian / learned opt. *(Lec 2–6)*
Dynamic, single	MDP / optimal control *(Lec 7, 9)*	reinforcement learning *(Lec 8, 10, 11)*

We begin at the **simplest cell**: one decision, a known objective, no time, no rivals. This is classical mathematical optimisation.

Why start here when the course is about *data-driven* decisions? Because this is the **atom**. Every later method — Bayesian optimisation, value iteration, policy gradient, trust-region RL — is this template with something made uncertain, sequential or sampled. Master the atom and the molecules become legible.

The thesis — formulation precedes solution

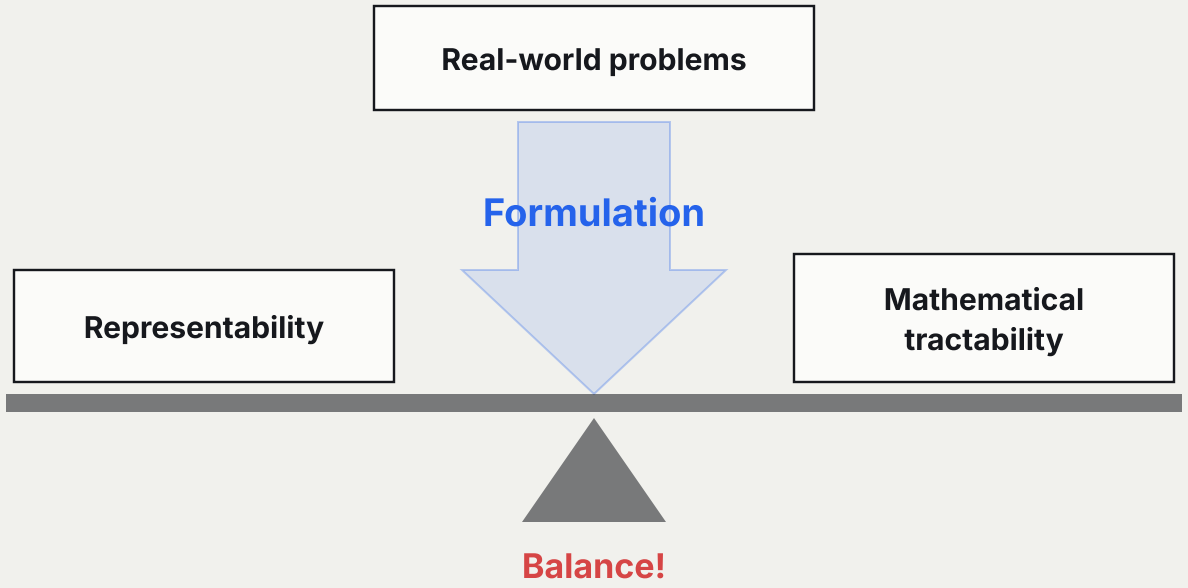
Before you can learn a decision, you must be able to state one.

A decision problem becomes mathematics through three ingredients — and naming them is half the work:

- an **objective** $f(x)$ — what "good" means, as a number to minimise;
- **decision variables** x — the levers you may pull;
- **constraints** $g(x) \leq 0,\ h(x) = 0$ — what is allowed.

Get this right and the rest is method. Get it wrong — the wrong objective, a missing constraint — and no solver, classical or learned, can save you. The hardest and most consequential step in applied decision making is the **modelling**, not the solving.

Formulation is a balance



A model that keeps every detail of the world cannot be solved; a model a solver loves may no longer be the problem you had. **Formulation is the act of balancing the two** — and it is where most of the real difficulty of applied decision making lives.

The roadmap — four questions

Q1 State it? → Q2 Solve it? → Q3 Certify it? → Q4 Convexify it?

- **Q1 — How do we state a decision mathematically?** The standard form.
- **Q2 — Which problems can we actually solve?** Convexity — the great watershed.
- **Q3 — How do we *know* a solution is optimal?** Optimality conditions and KKT.
- **Q4 — What about the non-convex ones?** Successive convexification and trust regions.

ACT 1

The standard form

Q1. One shape that every optimisation problem can be poured into.

The standard form — one shape for all of them

Q1 State it? → Q2 Solve it? → Q3 Certify it? → Q4 Convexify it?

Every mathematical optimisation problem can be written:

$$\min_{x \in D} \; f(x) \qquad \text{s.t.} \quad g_i(x) \le 0,\; i = 1, \dots, m, \qquad h_j(x) = 0,\; j = 1, \dots, p$$

- $x \in \mathbb{R}^n$ — the optimisation variable; $f : \mathbb{R}^n \to \mathbb{R}$ — the objective;
- g_i — inequality constraints; h_j — equality constraints;
- the **optimal value** $p^* = \inf\{f(x) : x \text{ feasible}\}$.

Two degenerate cases worth naming: $p^* = +\infty$ if the problem is *infeasible* (no x satisfies the constraints), and $p^* = -\infty$ if it is *unbounded below*. Both are usually signs of a modelling error, not a solver failure.

Maximising f is minimising $-f$; everything is phrased as minimisation without loss of generality.

Equivalent problems — four rewrites worth knowing

The standard form is a shape, and most problems must be *put into* it. Four transformations do almost all of that work, and each leaves the optimal value unchanged.

REWRITE	FROM	TO
equality constraints	$f(\mathbf{A}_i x + \mathbf{b}_i)$ inside	$f(z_i)$ with $z_i = \mathbf{A}_i x + \mathbf{b}_i$
slack variables	$a_i^\top x \le b_i$	$a_i^\top x + s_i = b_i,\ s \ge 0$
epigraph form	$\min_x f(x)$	$\min_{x,t} t$ s.t. $f(x) - t \le 0$
minimise out a variable	$\min_{x_1,x_2} f(x_1, x_2)$	$\min_{x_1} \hat{f}(x_1),\ \hat{f} = \inf_{x_2} f$

The **epigraph form** is the one to remember: every convex problem can be written with a *linear* objective, because the difficulty pushes into the single constraint $f(x) \le t$.

ACT 2

Convexity, the watershed

Q2. Of all the problems you can state, which can you actually solve?

Convexity — the line between easy and hard

Q1 State it? → **Q2 Solve it?** → Q3 Certify it? → Q4 Convexify it?

A problem is **convex** when f is a convex function and the feasible set is a convex set — g_i convex, h_j affine.

WHY CONVEXITY IS *THE* DIVIDING LINE

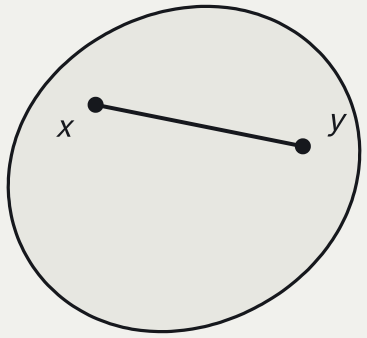
For a convex problem, **any local minimum is a global minimum.**

That single fact changes everything:

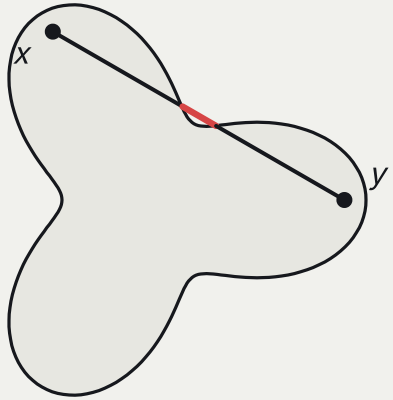
- a convex problem has a **globally** optimal solution, not merely a locally optimal one;
- reliable, efficient solvers exist;
- the choice of solver and its internal settings — initialisation, step size, batch size — **does not matter**;
- global optimality is **certifiable** — via KKT, in Act 3;
- the dual problem gives a computable lower bound and an optimality gap;
- distributed and decentralised methods are well studied.

Convex sets — the segment test

Non-convex problems enjoy none of those guarantees for free, so much of practical optimisation is the art of **getting a convex problem** — or a sequence of them. And both halves of one are the same test.



convex
every chord stays inside

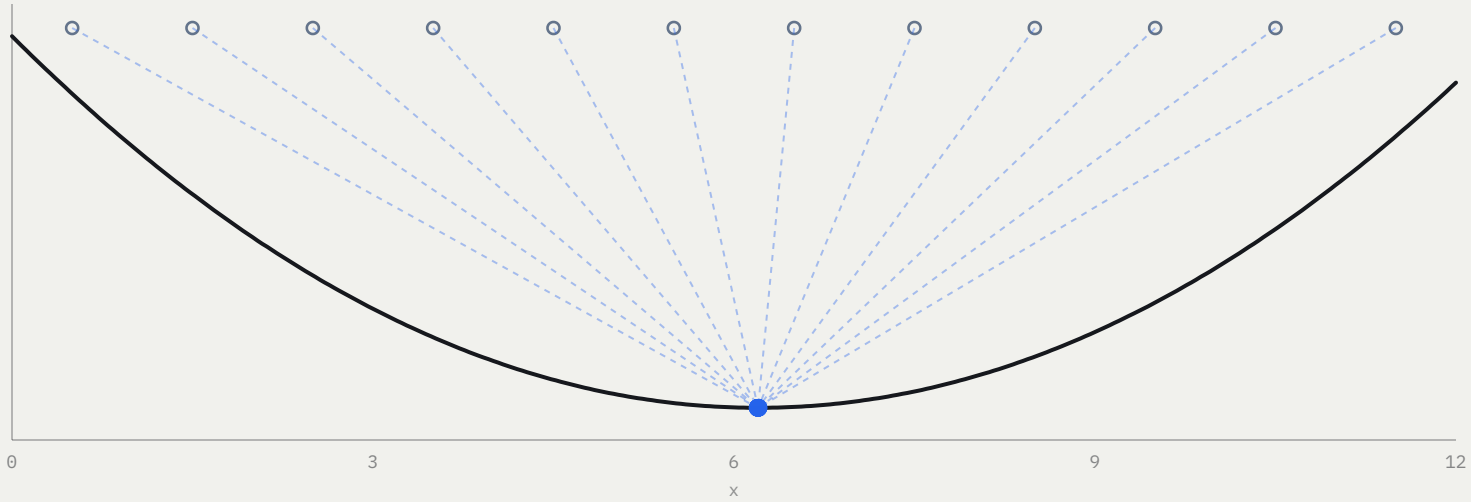


not convex
this chord leaves the set

A set is convex when the segment joining any two of its points stays inside it. A function is convex when the segment joining any two points of its graph stays *above* it — equivalently, when its epigraph is a convex set. **Both halves of a convex problem are this one test.**

The watershed, run twice

LOCAL DESCENT, TWELVE STARTS, ONE OBJECTIVE



every start reached the same point — local minimum = global minimum

The same descent rule, the same twelve starting points, two objectives. On the convex one the starting point is irrelevant; on the non-convex one it decides the answer. Everything in Act 2 follows from this single picture.

The convex family — LP, QP, QCQP

Named convex forms, in order of generality.

CLASS	OBJECTIVE	FEASIBLE SET
Linear program (LP)	$c^\top x + d$	a polyhedron (affine inequalities)
Quadratic program (QP)	$\frac{1}{2}x^\top P x + q^\top x$, with $P \succeq 0$	a polyhedron
QCQP	convex quadratic	an intersection of ellipsoids

The progression matters because modelling is often a matter of *recognising* which named class your problem — or its convexified version — falls into; each has mature, reliable solvers. The LP's optimum sits at a vertex of the polyhedron; the QP's, where the quadratic's level sets first touch the feasible region.

These same quadratic models reappear in Act 4 as the *local* approximation of hard problems — and, much later, as the trust-region subproblem inside TRPO (Lecture 10).

Three problems that are secretly linear programs

DIET *(EXAMPLE 1.2)*

Buy quantities x_j of n foods as cheaply as possible; food j costs c_j and carries a_{ij} of nutrient i, and the diet needs at least b_i of each.

$$\min_{x \in \mathbb{R}^n_+} c^\top x \quad \text{s.t.} \sum_j a_{ij} x_j \geq b_i$$

PIECEWISE-LINEAR *(EXAMPLE 1.3)*

A maximum of affine pieces is not linear, but its *epigraph* is:

$$\min_x \max_i (a_i^\top x + b_i)$$

$$= \min_{x,t} t \ \text{ s.t. } a_i^\top x + b_i \leq t$$

The epigraph rewrite of Act 1, earning its keep.

CHEBYSHEV CENTRE *(EXAMPLE 1.4)*

The centre of the largest ball inscribed in $\mathcal{P} = \{x : a_i^\top x \leq b_i\}$. The ball fits iff $\sup_{\|u\| \leq r} a_i^\top (x_c + u) \leq b_i$, and that supremum is available in closed form:

$$\max_{x_c, r} r \ \text{ s.t. } a_i^\top x_c + r\|a_i\|_2 \leq b_i$$

None of the three looks linear when stated. **Modelling is the act of finding the rewrite**, and it is where the expertise lives — the solver is a commodity.

Two that are quadratic, and one that is neither

QUADRATIC PROGRAMS

Least squares with bounds. $\min_x \|\mathbf{A}x - b\|_2^2$ subject to $l \le x \le u$ — a convex quadratic over a box.

A linear programme with random cost. If c has mean $\bar{c}$ and covariance Σ, then $c^\top x$ has mean $\bar{c}^\top x$ and variance $x^\top \Sigma x$, so

$$\min_x \ \bar{c}^\top x + \gamma\, x^\top \Sigma x$$

trades expected cost against risk. **γ is the first risk parameter of the course** — and the first time uncertainty enters an objective rather than a constraint.

ROBUST LINEAR PROGRAMMING

The parameters of a real problem are rarely known. With g_i uncertain there are two honest formulations, and they are the two halves of Lecture 0's uncertainty split:

Deterministic (worst case). The constraint must hold for *every* g_i in an uncertainty set $\mathcal{E}_i$:

$$g_i^\top x \le h_i \quad \forall g_i \in \mathcal{E}_i$$

Stochastic (chance constrained). g_i is random and the constraint need only hold with probability η:

$$\mathbf{P}\left(g_i^\top x \le h_i\right) \ge \eta$$

Both keep the problem convex for the usual choices of $\mathcal{E}_i$ and η — which is the point. **Robustness is bought inside the convex world, not outside it.** Lecture 2 will take the second reading much further, and Lecture 5 will meet the first again when a surrogate has to be trusted only where the data supports it.

Convexity, imposed on a learned model

pp. 20–24 of the source — where this lecture reaches into Part IV

The classes above assume someone hands you f. Modern practice does the opposite: it learns f from data and then wants to optimise over it — and a neural network is not convex in its input, so the resulting problem has none of the guarantees of this act.

- **Input Convex Neural Networks (ICNN)** — constrain the weights so the network's *output is a convex function of its input*, while remaining a free function of its parameters. Fit the model however you like; the control problem it induces is then convex. *(Amos, Xu & Kolter, 2017)*
- **Optimisation as a layer** — OptNet and differentiable convex layers put an $\arg\min$ *inside* a network and differentiate through it, using the **derivative of the KKT system** of Act 3. *(Amos & Kolter, 2017; Agrawal et al., 2019)*
- **Implicit deep learning** — a layer defined by a condition its output must satisfy rather than by a formula. Optimisation ($z^\star = \arg\min_z f_\theta(x, z)$), fixed points ($z^\star = f_\theta(x, z^\star)$) and neural ODEs are the same construction, trained through the implicit function theorem: $\dfrac{\partial \mathcal{L}}{\partial \theta} = \dfrac{\partial z^\star}{\partial \theta}\dfrac{\partial \mathcal{L}}{\partial z^\star}$.

This is the seam between Lecture 1 and Part IV. **Convexity is not only a property you find; it is one you can impose** — and Lecture 11 will impose it on a learned dynamics model so that a planner can differentiate through the control problem, in the professor's own bilevel design work.

ACT 3

Certifying optimality

Q3. A solver returns a point. How do you know it is *the* point?

When is a point optimal? — the first-order condition

Q1 State it? → Q2 Solve it? → **Q3 Certify it?** → Q4 Convexify it?

For a convex problem $\min_{x \in D} f(x)$ with differentiable f, a feasible x^* is optimal **if and only if**

$$\nabla f(x^*)^\top (y - x^*) \geq 0 \qquad \text{for all feasible } y$$

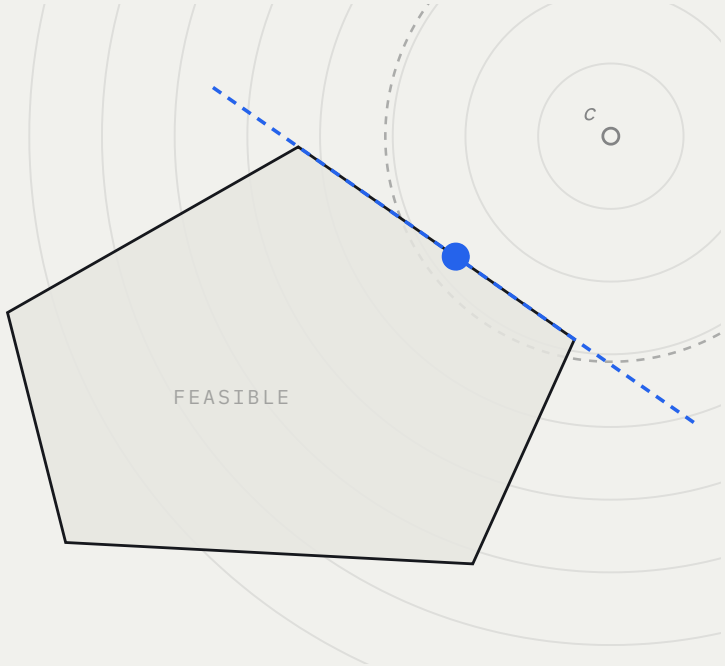
Reading it geometrically. Moving from x^* toward any other feasible point cannot decrease f — there is no improving feasible direction. If $\nabla f(x^*) \neq 0$ it defines a **supporting hyperplane** to the feasible set at x^*: the whole feasible region lies on the uphill side.

Optimality = **no feasible direction points downhill.**

For an *unconstrained* convex problem this collapses to the familiar $\nabla f(x^*) = 0$.

The condition, made draggable

DRAG THE POINT — IS ANY FEASIBLE DIRECTION DOWNHILL?



OPTIMAL

no feasible direction points downhill

x = (7.47, 6.65) · f(x) = 9.63

$\min_{y \text{ feasible}} \nabla f(x)^T (y - x) = \mathbf{0.00} \geq 0$

The unconstrained minimiser sits outside the feasible set, so the optimum is pushed onto the boundary. There the gradient is normal to the active edge and points out of the set — that normal, scaled, is the multiplier $\lambda \geq 0$ of KKT.

Minimise $\|x - c\|^2$ over a polygon, with c outside it. Drag the point and read the test literally: the red arrow is a feasible direction that decreases f. It disappears exactly at the optimum, where the gradient supports the set — and that supporting normal, scaled, **is the KKT multiplier**.

KKT — optimality with constraints, and a dual bound

With constraints, introduce multipliers $\lambda_i \geq 0$ for g_i and ν_j for h_j. The **Karush–Kuhn–Tucker** conditions characterise optimality:

$$\nabla f(x^*) + \sum_i \lambda_i \nabla g_i(x^*) + \sum_j \nu_j \nabla h_j(x^*) = 0, \qquad g_i(x^*) \leq 0, \;\; h_j(x^*) = 0, \;\; \lambda_i \geq 0, \;\; \lambda_i \, g_i(x^*) = 0$$

The last identity, **complementary slackness**, says each constraint is either active ($g_i = 0$) or ignored ($\lambda_i = 0$). For convex problems KKT is *necessary and sufficient* — a certificate of global optimality.

Duality, in one line. The Lagrangian $L(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i + \sum_j \nu_j h_j$ yields a dual function whose value **lower-bounds** p^* for any $\lambda \geq 0$. The gap between primal and dual is a computable measure of how close you are.

Differentiating the KKT system is exactly how one back-propagates through an optimisation layer — the trick behind differentiable LQR and MPC in Lecture 11.

ACT 4

When the problem is not convex

Q4. Real problems are not convex. Do we give up the guarantees, or manufacture them?

Successive convexification — solve a sequence of easy problems

Q1 State it? → Q2 Solve it? → Q3 Certify it? → **Q4 Convexify it?**

Real engineering problems are usually non-convex — non-convex objective, non-convex constraints. The dominant strategy: **don't solve the hard problem; solve a sequence of convex approximations to it.**

At the current iterate $x^{(k)}$, build a local convex model

$$\tilde{f}(x) = f(x^{(k)}) + \nabla f(x^{(k)})^\top (x - x^{(k)}) + \tfrac{1}{2}(x - x^{(k)})^\top B^{(k)} (x - x^{(k)})$$

linearise the awkward constraints, and add a **trust region** $\|x - x^{(k)}\| \le \rho^{(k)}$ so the model stays trustworthy.

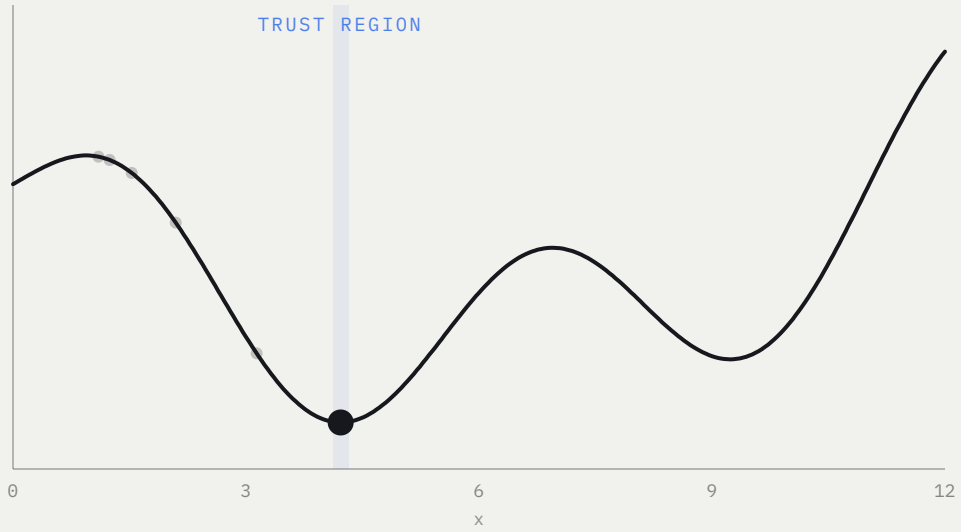
The trust region is the key idea: **only trust the approximation nearby.**

Solve the convex subproblem; if it improves the true objective, accept and *grow* ρ; if not, reject and *shrink* ρ. Repeat.

The trust region, iterated

SUCCESSIVE CONVEXIFICATION — A STAIRCASE OF EASY PROBLEMS

ITERATE **15**



CONVERGED

the gradient has vanished and the trust region has closed in

$x^{(k)}$ = **4.219** · f = **0.864**

trust radius ρ = **0.100** · gradient f′ = -0.000

accepted steps: 10 of 15

The ratio asks whether the convex model deserved to be believed. Trust grows where it predicts well and shrinks where it does not — nothing about the true objective is ever solved directly.

Step it and watch the ratio test do the work: an accepted step earns more trust, a rejected one halves it. The true objective is never solved — only a staircase of quadratics inside a shrinking and growing box. **This exact logic returns as TRPO's KL trust region in Lecture 10.**

Case study — wind-farm layout optimisation

the problem this lecture was built around

WHY THE PROBLEM IS HARD

A wind farm loses efficiency as it grows: the largest, the London Array, draws 630 MW from 175 turbines, and each turbine sits in the **wake** of those upwind of it. Power at turbine i therefore depends on where *every other* turbine is.

The wake deficit is the Park model,

$$\delta u(d, r) = 2\alpha \left(\frac{R_0}{R_0 + \kappa d} \right)^2 \exp\left(- \left(\frac{r}{R_0 + \kappa d} \right)^2 \right)$$

in the down-stream distance

d

and the radial distance

r

.

WHAT IS ACTUALLY OPTIMISED

Wind direction and speed are random — direction from the site's own rose, speed Weibull within each direction bin — so the objective is an **expectation** over their joint mass function:

$$\max_{l} \ \mathbb{E}\Big[\textstyle\sum_{i=1}^{N} P_i(l; U, \theta^W) \Big] \approx \sum_k \sum_j \sum_i P_i(l; U_j, \theta_k^W) \Pr(U_j, \theta_k^W)$$

$$\text{s.t.} \quad \|l_i - l_j\|_2 \geq 5D, \qquad \underline{c} \leq \mathbf{C} l \leq \bar{c}$$

The objective is a smooth expectation, but **the spacing constraint is not convex** — a minimum distance excludes a *ball*, and the complement of a ball is the wrong side of everything this lecture has built.

The staircase, in the professor's own algorithm

At iterate $l^{(k)}$: take the analytic gradient $\nabla f(l^{(k)})$ and an approximate Hessian $B^{(k)}$, **linearise** each spacing constraint about the current layout, and add a trust region.

$$\max_{l} \; \tilde{f}(l) = f(l^k) + \nabla f(l^k)^\top (l - l^k) + \tfrac{1}{2}(l - l^k)^\top B^{(k)} (l - l^k)$$

$$\text{s.t. } \left(l_i^{(k)} - l_j^{(k)}\right)^\top (l_i - l_j) > 4D \left\| l_i^{(k)} - l_j^{(k)} \right\|_2, \qquad l \in T^{(k)} = \{ l : \| l - l^k \| < \rho^{(k)} \}$$

ALGORITHM 1 — LAYOUT OPTIMISATION BY SUCCESSIVE CONVEX PROGRAMMING

Solve the convex subproblem for $\tilde{l}$, then judge it by the ratio $\dfrac{f(l^{(k)}) - f(\tilde{l})}{f(l^{(k)}) - \tilde{f}(\tilde{l})} \geq a$ — **accept and grow** $\rho^{(k+1)} = \beta^{\text{succ}} \rho^{(k)}$, otherwise **reject and shrink** $\rho^{(k+1)} = \beta^{\text{fail}} \rho^{(k)}$.

Repeat until $\| l^{(k)} - l^{(k-1)} \| < \epsilon$.

That ratio test is exactly what the widget two slides ago was running. **The toy on that slide is this algorithm.** Run on a real farm it lifts power efficiency from about 0.69 to 0.76 and flattens the efficiency-versus-direction curve, so the farm is not only better on average but steadier. The non-convex layout problem is conquered by **a staircase of convex problems** — Act 2's lesson, made operational: *get a convex problem, even if you have to keep making new ones.*

CLOSING

Closing

We can state a decision and, when it is convex, prove we have solved it. Now the assumption we drop.

Where we are — and the assumption we now drop

	MODEL-BASED	DATA-DRIVEN
Static, single	optimisation *(Lec 1 ✓)*	Bayesian / learned opt. *(Lec 2–6 →)*

We can now *state* a decision and, when it is convex or convexifiable, *solve and certify* it. But every line above assumed one thing:

the objective f and the constraints were known, **exactly.**

What if f has *uncertain parameters* — a model fitted to noisy data? Then a single best x is not enough; we must reason about our **uncertainty about f itself**. That is Lecture 2: don't pick a number — carry a belief.

Optimization is how a decision becomes mathematics — and, when the problem is convex, an answer we can not only find but *prove*.

An objective to minimise, levers to pull, limits to respect.

Questions?

Hold onto two pieces: the standard form — it is the skeleton of every objective to come — and the trust region, which returns in surrogate optimisation and again in trust-region RL. Everything else this term is this atom, perturbed by uncertainty.

APPENDIX

Backup slides

Complete statements, kept out of the narrative.

Backup 1 — the KKT conditions in full

For $\min f(x)$ subject to $g_i(x) \le 0$ and $h_j(x) = 0$, the KKT conditions at x^*, with multipliers $\lambda^* \ge 0$ and ν^*:

- **Stationarity** — $\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0$
- **Primal feasibility** — $g_i(x^*) \le 0$, $h_j(x^*) = 0$
- **Dual feasibility** — $\lambda_i^* \ge 0$
- **Complementary slackness** — $\lambda_i^* \, g_i(x^*) = 0$

Convex case. If f, g_i are convex and h_j affine, and a constraint qualification such as Slater's holds, KKT is *necessary and sufficient* for global optimality — a checkable certificate. **Non-convex case.** KKT remains necessary under a constraint qualification but not sufficient: a KKT point may be a local minimum, a saddle, or a local maximum.

Backup 2 — Lagrangian duality and the gap

Lagrangian. $L(x, \lambda, \nu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \nu_j h_j(x)$.

Dual function. $d(\lambda, \nu) = \inf_x L(x, \lambda, \nu)$. For any $\lambda \geq 0$ and any ν, weak duality gives $d(\lambda, \nu) \leq p^*$ — the dual is a *lower bound* on the optimum, always.

Dual problem. $\max_{\lambda \geq 0,\, \nu} d(\lambda, \nu) =: d^*$. The **duality gap** is $p^* - d^* \geq 0$. For convex problems with a constraint qualification the gap is zero ($p^* = d^*$, *strong duality*), so the dual optimum certifies the primal.

Use. Even when you cannot solve the primal, evaluating $d(\lambda, \nu)$ at any feasible dual point brackets how far your current solution can possibly be from optimal — the practical value of duality.

Backup 3 — the successive convex programming loop

Algorithm — trust-region SCP.

1. Initialise $x^{(0)}$ and a trust radius $\rho^{(0)}$.
2. **Repeat** until $\|x^{(k)} - x^{(k-1)}\| < \epsilon$:
3. form the convex model $\tilde{f}$ — gradient plus an approximate Hessian $B^{(k)}$ — and linearise the constraints;
4. solve the convex subproblem over $\{x : \|x - x^{(k)}\| \le \rho^{(k)}\}$, giving a candidate $\tilde{x}$;
5. compute the ratio $r = \dfrac{f(x^{(k)}) - f(\tilde{x})}{f(x^{(k)}) - \tilde{f}(\tilde{x})}$ — actual over predicted improvement;
6. if $r \ge a$: accept $x^{(k+1)} = \tilde{x}$ and grow $\rho^{(k+1)} = \beta_{\text{succ}}\, \rho^{(k)}$;
7. else: reject $x^{(k+1)} = x^{(k)}$ and shrink $\rho^{(k+1)} = \beta_{\text{fail}}\, \rho^{(k)}$.

Why the ratio test. It measures whether the convex model can be trusted at the proposed step. Trust grows where the model predicts well and shrinks where it does not — the exact logic that, in policy space, becomes TRPO's KL trust region in Lecture 10.